

Blocco 14 - Scenari e modellazione NoSQL

Dal requisito applicativo al modello dati non relazionale

Relational Databases & SQL

30 min teoria + 90 min pratica

1. Modellare partendo dalle query

Nel relazionale chiediamo spesso “quali entità esistono?”. In NoSQL chiediamo prima “come leggerò e scriverò il dato?”.

Obiettivo del blocco

Problema

Capire come si passa da scenario applicativo a modello NoSQL senza cadere in documenti casuali o duplicazioni non governate.

Idea guida

In NoSQL il modello è corretto se serve bene le query principali e rende espliciti i compromessi.

Un access pattern descrive una domanda concreta:

- ▶ leggere l'ultima misura di un dispositivo;
- ▶ mostrare il trend degli ultimi 30 minuti;
- ▶ trovare dispositivi in allarme;
- ▶ calcolare media temperatura per sito;
- ▶ cancellare dati più vecchi di 90 giorni.

Senza access pattern si rischia di progettare un modello elegante ma poco utilizzabile.

Esempio: richiesta informale

Richiesta

“Vogliamo monitorare sensori energetici e ambientali e vedere subito anomalie e trend.”

Questa frase deve diventare:

- ▶ tipi di dispositivo;
- ▶ forma dei messaggi;
- ▶ frequenza di arrivo;
- ▶ query di dashboard;
- ▶ soglie di alert;
- ▶ politica di conservazione.

Voce	Decisione
Unità dato	lettura telemetrica ricevuta dal collector
Formato	documento con metadati e oggetto <code>metrics</code> variabile
Query veloci	ultime letture, trend, warning, aggregazioni per sito
Coerenza	eventuale per dashboard, forte solo per configurazione dispositivi
Retention	raw breve, curated più lunga, aggregati storici

2. Document modelling

Il documento non è una tabella senza regole: è un aggregate disegnato per letture e aggiornamenti specifici.

Embedding o reference?

Embedding

Mettere dati annidati nello stesso documento. Utile quando vengono letti insieme e aggiornati insieme.

Reference

Salvare un identificativo verso un altro documento. Utile quando l'oggetto collegato è grande, condiviso o aggiornato separatamente.

La domanda pratica è: “se cambio questo pezzo, quanti documenti devo aggiornare e quanto spesso?”

Esempio: dispositivo e letture

- ▶ il dispositivo ha metadati stabili: sito, regione, tipo;
- ▶ le letture arrivano molte volte al minuto;
- ▶ incorporare tutte le letture nel documento dispositivo lo farebbe crescere senza controllo;
- ▶ meglio separare `devices` e `readings_curated`;
- ▶ nella lettura possiamo duplicare uno snapshot di sito e regione per accelerare la dashboard.

Documento dispositivo

```
{  
  "device_id": "ENV-torino-004",  
  "device_type": "air_quality_station",  
  "site": "Torino Park",  
  "region": "north",  
  "active": true,  
  "tags": ["environment", "air_quality"]  
}
```

```
{
  "device_id": "ENV-torino-004",
  "device_type": "air_quality_station",
  "site": "Torino Park",
  "region": "north",
  "ts": ISODate("2026-05-13T08:10:00Z"),
  "metrics": {
    "temperature_c": 20.1,
    "humidity_pct": 60,
    "pm25_ugm3": 37
  },
  "status": "warning"
}
```

In NoSQL duplicare non è automaticamente un errore.

- ▶ duplicare `site` e `region` in ogni lettura evita lookup ripetuti;
- ▶ se il sito cambia nome, bisogna decidere se correggere lo storico;
- ▶ per telemetria spesso lo snapshot storico è accettabile;
- ▶ per dati anagrafici critici la duplicazione può diventare pericolosa.

La duplicazione deve essere una scelta documentata, non un incidente.

3. Indici e chiavi

Un modello NoSQL senza indici coerenti con le query è solo un archivio di documenti.

Indice come promessa di accesso

Per la telemetria servono almeno:

- ▶ `device_id + ts`: ultima lettura e storia del dispositivo;
- ▶ `device_type + ts`: confronti per categoria;
- ▶ `region + ts`: dashboard territoriali;
- ▶ `severity + ts`: alert recenti.

Ogni indice accelera letture ma costa spazio e tempo in scrittura.

Indici MongoDB

```
db.readings_curated.createIndex({ device_id: 1, ts: -1 });  
db.readings_curated.createIndex({ device_type: 1, ts: -1 });  
db.readings_curated.createIndex({ region: 1, ts: -1 });  
db.alerts.createIndex({ severity: 1, ts: -1 });
```

- ▶ una chiave con pochi valori può concentrare troppe scritture;
- ▶ una chiave temporale pura può creare coda sull'ultima partizione;
- ▶ una chiave casuale distribuisce ma rende più difficili certe query;
- ▶ una chiave composta bilancia accesso e distribuzione.

Nei sistemi distribuiti la chiave non è solo un indice: determina anche dove vive il dato.

4. Aggregation pipeline

Le aggregazioni NoSQL spesso sono pipeline: ogni stadio trasforma il flusso di documenti.

Pipeline mentale

- ➊ **match**: scelgo i documenti;
- ➋ **project**: tengo o calcolo campi;
- ➌ **group**: aggrego;
- ➍ **sort**: ordino;
- ➎ **limit**: riduco il risultato.

È simile a una query SQL vista come sequenza: WHERE, colonne calcolate, GROUP BY, ORDER BY.

KPI warning

```
db.readings_curated.aggregate([
  {
    $group: {
      _id: null,
      readings: { $sum: 1 },
      devices: { $addToSet: "$device_id" },
      warnings: {
        $sum: { $cond: [{ $eq: ["$status", "warning"] }, 1, 0] }
      }
    }
  },
  { $project: { _id: 0, readings: 1, device_count: { $size: "$devices" }, warnings: 1 } }
]);
```

Trend temporale

```
db.readings_curated.aggregate([
  {
    $group: {
      _id: {
        bucket: {
          $dateTrunc: { date: "$ts", unit: "minute", binSize: 5 }
        }
      },
      readings: { $sum: 1 },
      warnings: { $sum: { $cond: [{ $eq: ["$status", "warning"] }, 1, 0] } }
    }
  },
  { $sort: { "_id.bucket": 1 } }
]);
```

5. Casi applicativi

Una tecnologia si capisce quando è collegata a uno scenario reale e ai suoi vincoli.

Caso 1: ticketing

- ▶ ticket core: relazionale, perché servono stato, integrità, dashboard SQL;
- ▶ eventi grezzi o allegati variabili: possibile documento;
- ▶ ricerca full text su descrizioni: possibile motore search;
- ▶ cache KPI: possibile key-value.

Lo stesso dominio può usare più storage, ognuno con responsabilità diversa.

Caso 2: energia

- ▶ sensori con metriche diverse;
- ▶ scritture append-only frequenti;
- ▶ aggregazioni per fascia oraria e sito;
- ▶ alert su soglie;
- ▶ retention differenziata tra raw e dati curati.

Document/time-series è una scelta plausibile, ma i dati master di impianti e contratti restano spesso relazionali.

Caso 3: monitoraggio ambientale

- ▶ centraline con sensori diversi;
- ▶ dati mancanti o rumorosi;
- ▶ coordinate, regione, area urbana;
- ▶ dashboard per PM2.5, vento, temperatura e umidità;
- ▶ controlli di qualità e calibrazione.

Caso 4: log applicativi

- ▶ volume alto;
- ▶ formato semi-strutturato;
- ▶ ricerca per testo, servizio, trace id;
- ▶ analisi temporale;
- ▶ retention corta.

Un motore search/log analytics può essere più adatto di un database general purpose.

6. Pratica guidata

La pratica consiste nel trasformare scenari informali in scelte motivate e modelli interrogabili.

Per ogni scenario:

- 1 descrivere il dato principale;
- 2 elencare tre query frequenti;
- 3 scegliere una famiglia NoSQL;
- 4 proporre un documento o una chiave;
- 5 indicare un indice;
- 6 dichiarare un rischio.

- ▶ la tecnologia è collegata a una query reale;
- ▶ il modello non è una copia meccanica del relazionale;
- ▶ duplicazioni e consistenza sono discusse;
- ▶ gli indici hanno una motivazione;
- ▶ i limiti sono espliciti.

- ▶ In NoSQL si modella partendo da access pattern e garanzie.
- ▶ Embedding e reference sono scelte operative.
- ▶ Gli indici devono seguire query e carico.
- ▶ I casi energia e ambiente preparano il sistema di telemetria dei blocchi 15-16.